

LAPORAN PRAKTIKUM

PEMROGRAMAN BERBASIS OBJEK

MEMBUAT FUNGSI CRUD (Create, Read, Update, Delete) USER DENGAN DATABASE MYSQL



OLEH:

DEVINA AMANDA PUTRI

(2411533009)

DOSEN PENGAMPU:

NURFIAH, S.ST, M.KOM

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

A. PENDAHULUAN

Pemrograman Berorientasi Objek (PBO) merupakan ilmu pemrograman yang berfokus pada konsep objek, di mana setiap objek memiliki atribut dan metode. Ilmu ini membantu developer membangun aplikasi yang lebih terstruktur, modular, dan mudah dikelola. Dalam praktikum kali ini, konsep PBO diterapkan dalam pembuatan aplikasi laundry yang mengimplementasikan operasi dasar CRUD.

CRUD (Create, Read, Update, Delete) merupakan empat operasi dasar dalam manajemen data, operasi ini adalah dasar dalam sebuah aplikasi yang mana operasi ini bisa untuk membuat, membaca, mengubah dan menghapus suatu data pada database aplikasi. Untuk mendukung proses tersebut, digunakan beberapa komponen penting. Xampp berperan sebagai lingkungan pengembangan lokal yang terdiri dari Apache, MySQL, PHP, dan Perl. Dalam konteks PBO berbasis Java, koneksi ke database MySQL dilakukan menggunakan MySQL Connector/J, sebuah driver yang memungkinkan aplikasi berinteraksi langsung dengan database. Selain itu, digunakan pula pendekatan DAO (Data Access Object) untuk memisahkan logika bisnis dengan logika akses data sehingga kode lebih modular, reusable, dan mudah dikembangkan. Konsep Interface dalam Java juga dimanfaatkan untuk mendefinisikan kontrak method yang wajib diimplementasikan oleh class terkait, sehingga struktur program lebih konsisten dan terorganisir.

Dengan menggabungkan konsep PBO dan implementasi CRUD, dapat dipahami cara pengolahan data dalam aplikasi dan juga mempraktikkan bagaimana prinsip modularitas, enkapsulasi, dan reusabilitas diterapkan dalam pembangunan software.

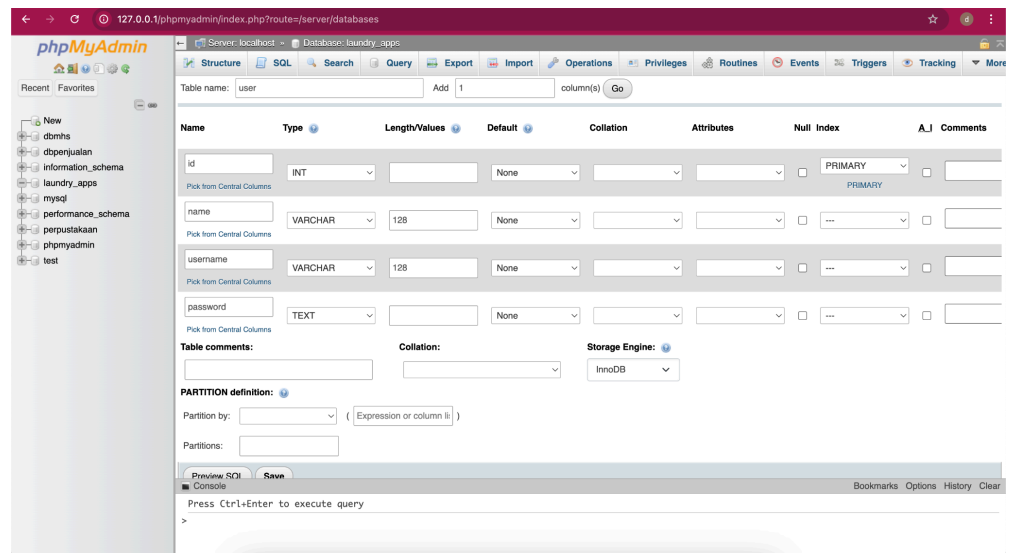
B. TUJUAN

Tujuan praktikum ini yaitu mahasiswa mampu membuat fungsi CRUD data user menggunakan database MySQL, Adapun poin-poin praktikum yaitu :

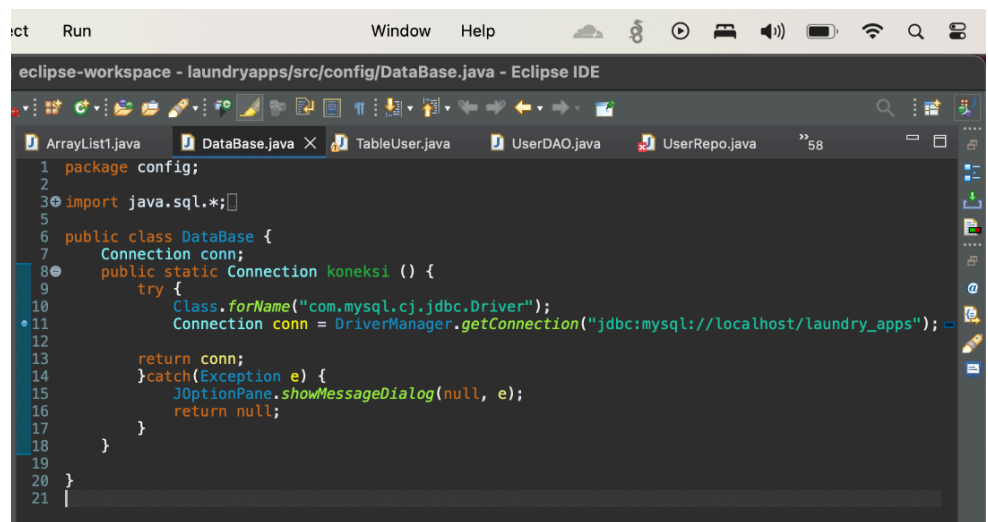
- Mahasiswa mampu membuat tabel user pada database MySQL
- Mahasiswa mampu membuat koneksi Java dengan database MySQL
- Mahasiswa mampu membuat tampilan GUI CRUD user
- Mahasiswa mampu membuat dan mengimplementasikan interface
- Mahasiswa mampu membuat fungsi DAO (Data Access Object) dan mengimplementasikan nya.
- Mahasiswa mampu membuat fungsi CRUD dengan menggunakan konsep Pemrograman Berorientasi Objek

C. LANGKAH KERJA

- Install MySQL connector dan menambahkan connector ke package project.
- Membuat database dan table user pada <http://localhost/phpmyadmin>, klik new dan buat database dengan nama laundry_apps, lalu buat tabel user, masukkan nama kolom dan tipe data nya.



- Koneksi Database MySQL
Setelah berhasil menambahkan MySQL Connector, buat koneksi ke database dengan membuat package baru dengan nama config.java, lalu buat class baru dengan nama Database.java, isi class tersebut dengan kode berikut:

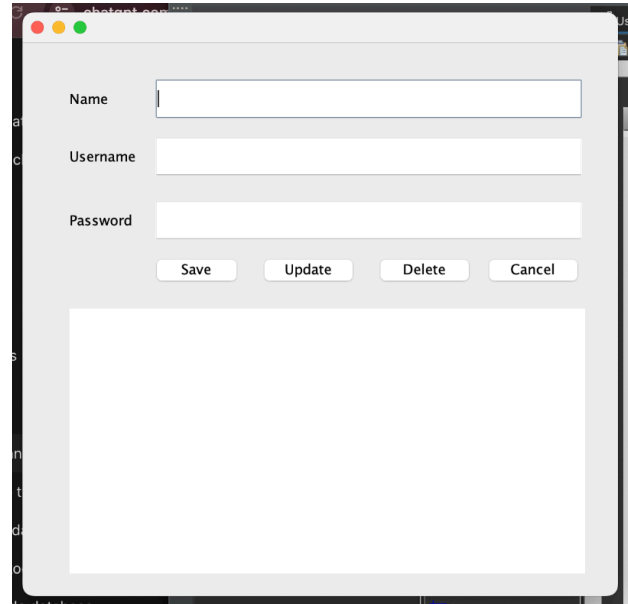


Dimana, Import java.sql.* untuk import semua fungsi SQL, method Connection digunakan untuk membuka koneksi ke database, dilanjut membuat koneksi database, jika koneksi berhasil akan mengembalikan nilai Connection, jika gagal akan muncul pesan error.

d. Tampilan CRUD

Buat file JFrame baru pada package ui, beri nama UserFrame. Buat tampilan dengan komponen:

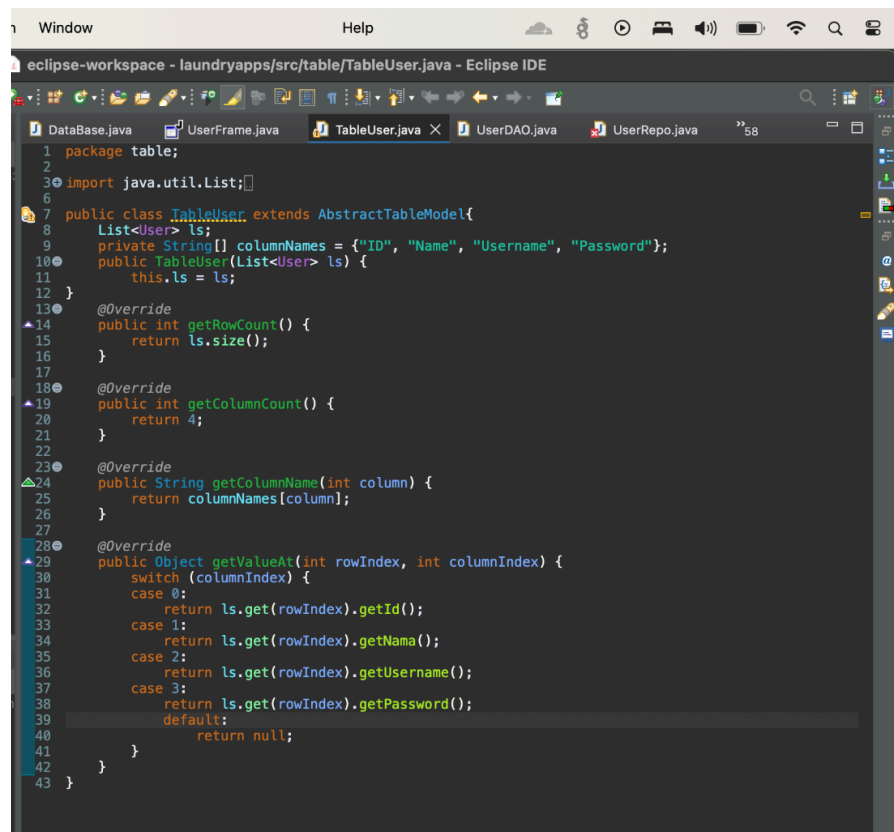
- JTextField dengan variabel txtName dan keterangan Name, txtUsername dengan keterangan Username, txtPassword dengan keterangan Password.
- JButton dengan variabel btnSave, btnUpdate, btnDelete, btnCancel.
- JTable dengan variabel tableUser dan keterangan Table Users.



e. Membuat Table Model

Tabel model user yang berguna untuk mengambil database dan ditampilkan ke dalam tabel. Buat package baru dengan nama table dan

class TableUser, ketikkan kode berikut:



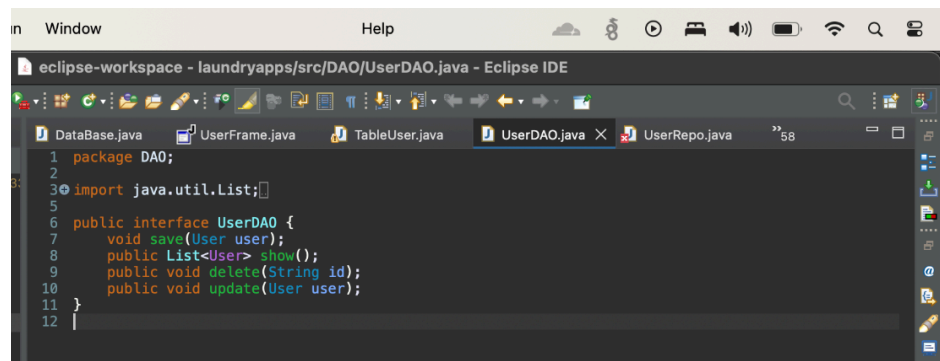
```
1 package table;
2
3 import java.util.List;
4
5
6
7 public class TableUser extends AbstractTableModel{
8     List<User> ls;
9     private String[] columnNames = {"ID", "Name", "Username", "Password"};
10    public TableUser(List<User> ls) {
11        this.ls = ls;
12    }
13
14    @Override
15    public int getRowCount() {
16        return ls.size();
17    }
18
19    @Override
20    public int getColumnCount() {
21        return 4;
22    }
23
24    @Override
25    public String getColumnName(int column) {
26        return columnNames[column];
27    }
28
29    @Override
30    public Object getValueAt(int rowIndex, int columnIndex) {
31        switch (columnIndex) {
32            case 0:
33                return ls.get(rowIndex).getId();
34            case 1:
35                return ls.get(rowIndex).getNama();
36            case 2:
37                return ls.get(rowIndex).getUsername();
38            case 3:
39                return ls.get(rowIndex).getPassword();
40            default:
41                return null;
42        }
43    }
44 }
```

1. Class TableUser dibuat dengan extend AbstractTableModel agar bisa digunakan sebagai model data untuk JTable.
2. Variabel ls bertipe List<User> digunakan sebagai tempat penyimpanan semua objek user yang akan ditampilkan.
3. Array columnNames berisi nama nama kolom tabel yaitu, ID, Name, Username dan Password.
4. Konstruktor TableUser menerima parameter berupa daftar user dan menyimpan ke variabel ls.
5. Method getRowCount() mengembalikan jumlah baris tabel sesuai ukuran list ls, method getColumnCount() mengembalikan jumlah kolom yang sesuai dengan array columnNames. Method getColumnName(int column) mengembalikan nama kolom sesuai index dan method getValueAt digunakan untuk mengambil nilai setiap sel di tabel.

f. Membuat Fungsi DAO

Buat package baru dengan nama DAO dan buat class interface

UserDAO. Isi dengan kode berikut:



```
1 package DAO;
2
3 import java.util.List;
4
5
6 public interface UserDAO {
7     void save(User user);
8     public List<User> show();
9     public void delete(String id);
10    public void update(User user);
11 }
12
```

1. UserDAO adalah interface yang mendefinisikan aturan dasar operasi CRUD pada tabel user.
 2. Method `save(User user)` digunakan untuk menyimpan data baru, method `show()` mengembalikan daftar semua data user dalam bentuk `List<user>`. Method `delete()` untuk menghapus data user berdasarkan ID dan method `Update()` untuk memperbarui data user yang sudah ada.
- g. Menggunakan Fungsi DAO
- Buat class baru pada package DAO dengan nama `UserRepo` yang akan digunakan untuk implementasi DAO yang sudah dibuat. Isikan dengan kode berikut

```
1 package DAO;
2
3 import java.util.logging.Level;
4 import java.util.logging.Logger;
5 import java.sql.Connection;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 import java.util.ArrayList;
11 import java.util.List;
12 import config.DataBase;
13 import model.User;
14
15 public class UserRepo implements UserDao {
16
17     private Connection connection;
18     final String insert = "INSERT INTO user (name, username, password) VALUES (?, ?, ?);";
19     final String select = "SELECT * FROM user;";
20     final String delete = "DELETE FROM user WHERE id=?;";
21     final String update = "UPDATE user SET name=?, username=?, password=? WHERE id=?;";
22
23     public UserRepo() {
24         connection = DataBase.koneksi();
25     }
26
27     @Override
28     public void save(User user) {
29         PreparedStatement st = null;
30         try {
31             st = connection.prepareStatement(insert);
32             st.setString(1, user.getName());
33             st.setString(2, user.getUsername());
34             st.setString(3, user.getPassword());
35             st.executeUpdate();
36         } catch (SQLException e) {
37             e.printStackTrace();
38         } finally {
39             try {
40                 if (st != null) st.close();
41             } catch (SQLException e) {
42                 e.printStackTrace();
43             }
44         }
45     }
46
47     @Override
48     public List<User> show() {
49         List<User> ls = null;
50         try {
51             ls = new ArrayList<User>();
52             Statement st = connection.createStatement();
53             ResultSet rs = st.executeQuery(select);
54             while (rs.next()) {
55                 User user = new User();
56                 user.setId(rs.getString("id"));
57                 user.setName(rs.getString("name"));
58                 user.setUsername(rs.getString("username"));
59                 user.setPassword(rs.getString("password"));
60                 ls.add(user);
61             }
62         } catch (SQLException e) {
63             Logger.getLogger(UserDAO.class.getName()).log(Level.SEVERE, null, e);
64         }
65         return ls;
66     }
67
68     @Override
69     public void update(User user) {
70         PreparedStatement st = null;
71         try {
72             st = connection.prepareStatement(update);
73             st.setString(1, user.getName());
74             st.setString(2, user.getUsername());
75             st.setString(3, user.getPassword());
76             st.setString(4, user.getId());
77             st.executeUpdate();
78         } catch (SQLException e) {
79             e.printStackTrace();
80         } finally {
81             try {
82                 if (st != null) st.close();
83             } catch (SQLException e) {
84                 e.printStackTrace();
85             }
86         }
87     }
88
89     @Override
90     public void delete(String id) {
91         PreparedStatement st = null;
92         try {
93             st = connection.prepareStatement(delete);
94             st.setString(1, id);
95             st.executeUpdate();
96         } catch (SQLException e) {
97             e.printStackTrace();
98         } finally {
99             try {
100                 if (st != null) st.close();
101             } catch (SQLException e) {
102                 e.printStackTrace();
103             }
104         }
105     }
106
107 }
```

1. Variable connection menyimpan objek koneksi yang diperoleh dari DataBase.koneksi().
2. Terdapat beberapa query SQL untuk memanipulasi data.
3. Pada konstruktor UserRepo(), koneksi database diinisialisasi agar bisa digunakan pada semua method CRUD

4. Method `save(user user)` menggunakan `PreparedStatement` untuk menjalankan query `INSERT`. Data diambil dari objek `User` lalu disimpan ke database.
5. Method `show()` menggunakan `Statement` untuk menjalankan query `SELECT`, hasilnya berupa `ResultSet` yang diolah baris per baris menjadi objek `User`, lalu ditambahkan ke `List<User>` dan dikembalikan.
6. Method `update(User user)` menjalankan query `UPDATE` dengan `PreparedStatement`, data lama akan diperbarui sesuai nilai baru pada objek `User`.
7. Method `delete` menjalankan query `DELETE` menggunakan `PreparedStatement` dengan parameter `ID`. Setelah dijalankan, data dengan `ID` tersebut akan terhapus dari database.

D. KESIMPULAN

Dari praktikum ini dapat disimpulkan bahwa Pemrograman Berbasis Objek (PBO) memberikan pemahaman mendasar mengenai pengelolaan data dalam sebuah aplikasi. Konsep DAO membantu memisahkan logika akses data dari logika bisnis, sementara interface memberi kerangka dasar agar implementasi lebih konsisten. Integrasi dengan MySQL Connector memungkinkan aplikasi Java berinteraksi dengan database secara efektif, mulai dari penyimpanan hingga penghapusan data.